

```
In [1]: import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [4]: df = pd.read_csv("diabetes .csv")
df.head()
```

```
Out[4]:
```

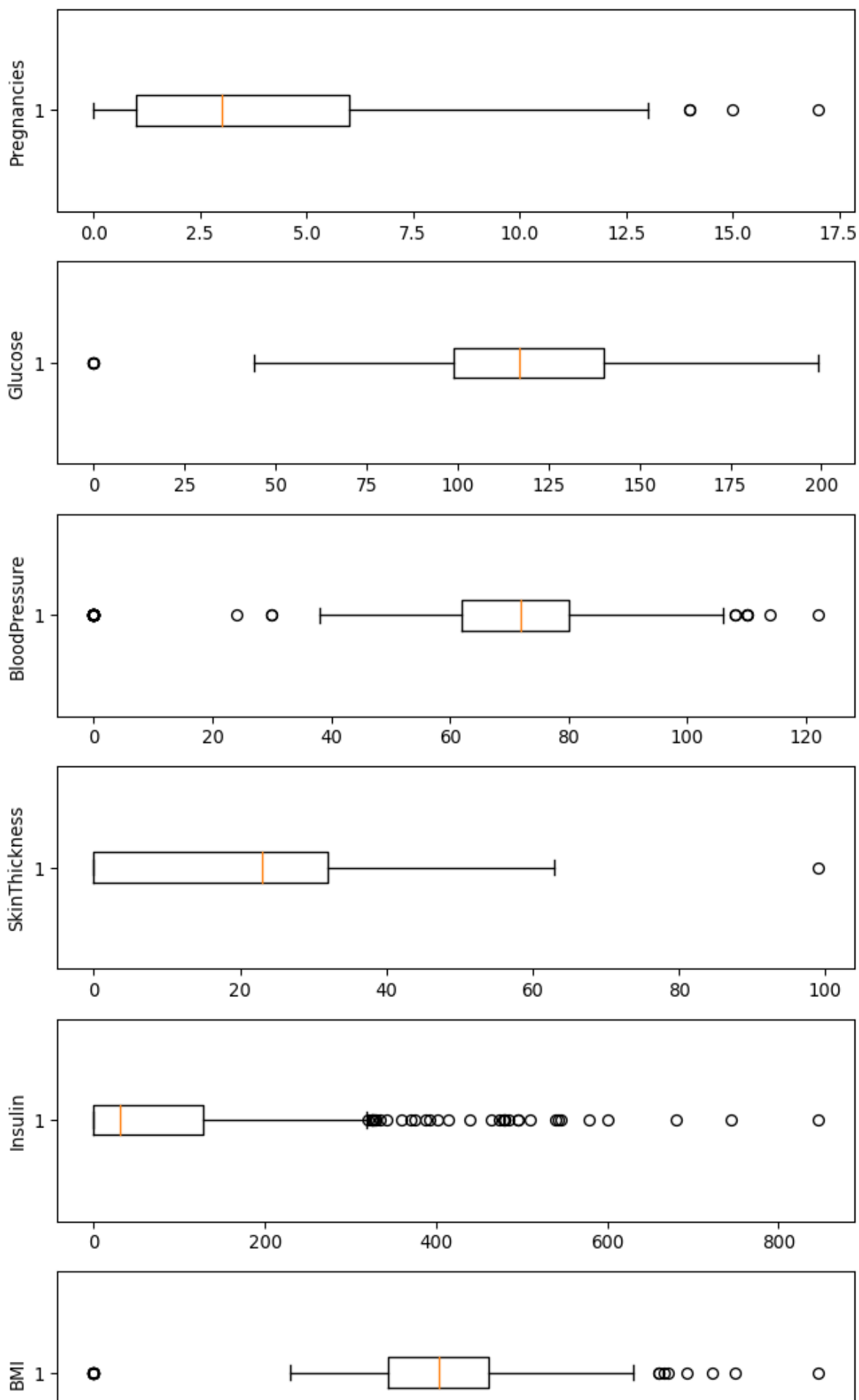
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72	35	0	33.6	0.627	
1	1	85	66	29	0	26.6	0.351	
2	8	183	64	0	0	23.3	0.672	
3	1	89	66	23	94	28.1	0.167	
4	0	137	40	35	168	43.1	2.288	

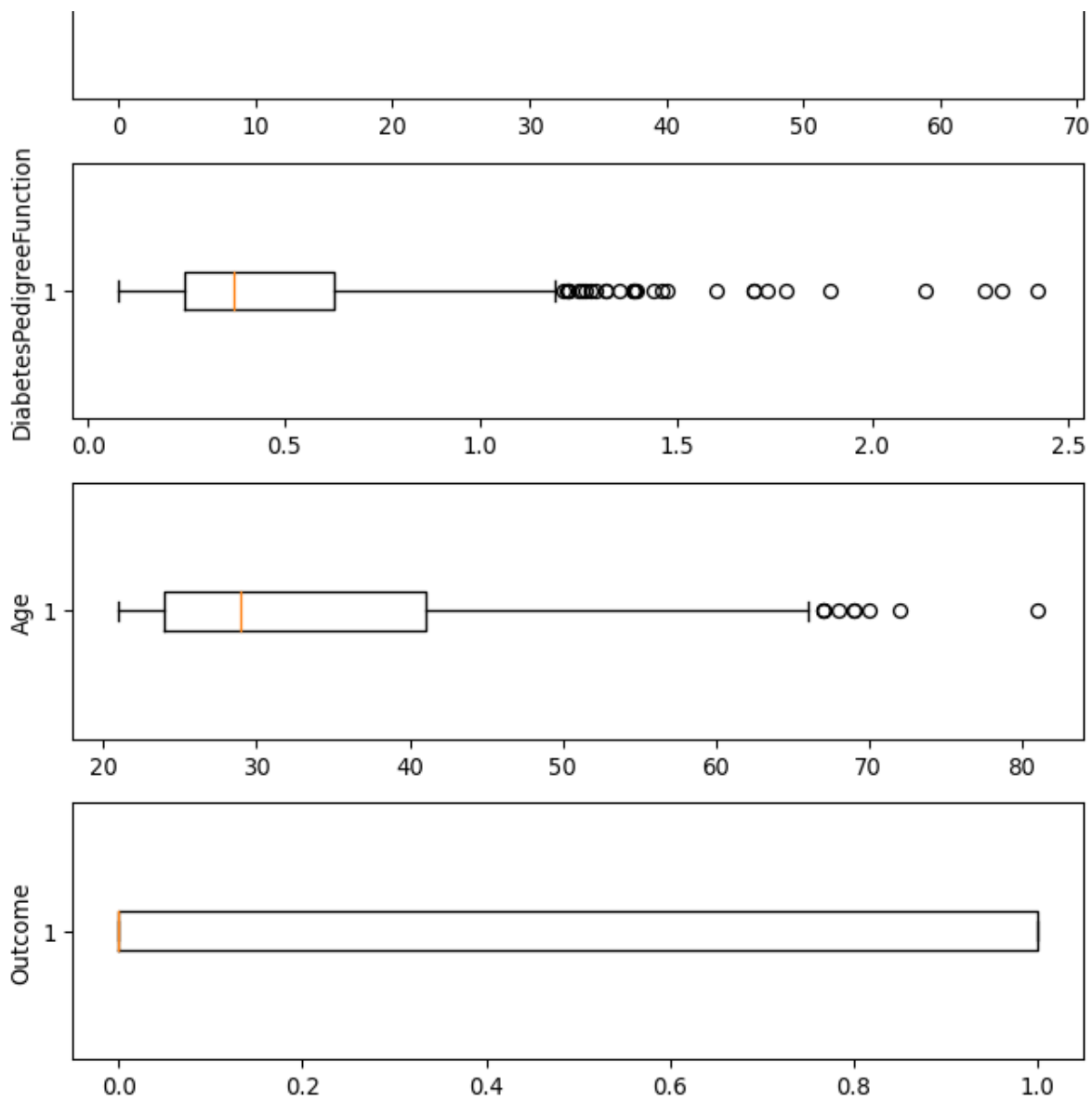
```
In [5]: df.info()
print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Pregnancies           768 non-null    int64
1   Glucose               768 non-null    int64
2   BloodPressure         768 non-null    int64
3   SkinThickness         768 non-null    int64
4   Insulin               768 non-null    int64
5   BMI                  768 non-null    float64
6   DiabetesPedigreeFunction 768 non-null    float64
7   Age                  768 non-null    int64
8   Outcome              768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
Pregnancies           0
Glucose               0
BloodPressure         0
SkinThickness         0
Insulin               0
BMI                   0
DiabetesPedigreeFunction 0
Age                   0
Outcome               0
dtype: int64
```

```
In [6]: df.describe()

fig, axs = plt.subplots(len(df.columns), 1, figsize=(7, 18), dpi=95)
for i, col in enumerate(df.columns):
    axs[i].boxplot(df[col], vert=False)
    axs[i].set_ylabel(col)
plt.tight_layout()
plt.show()
```

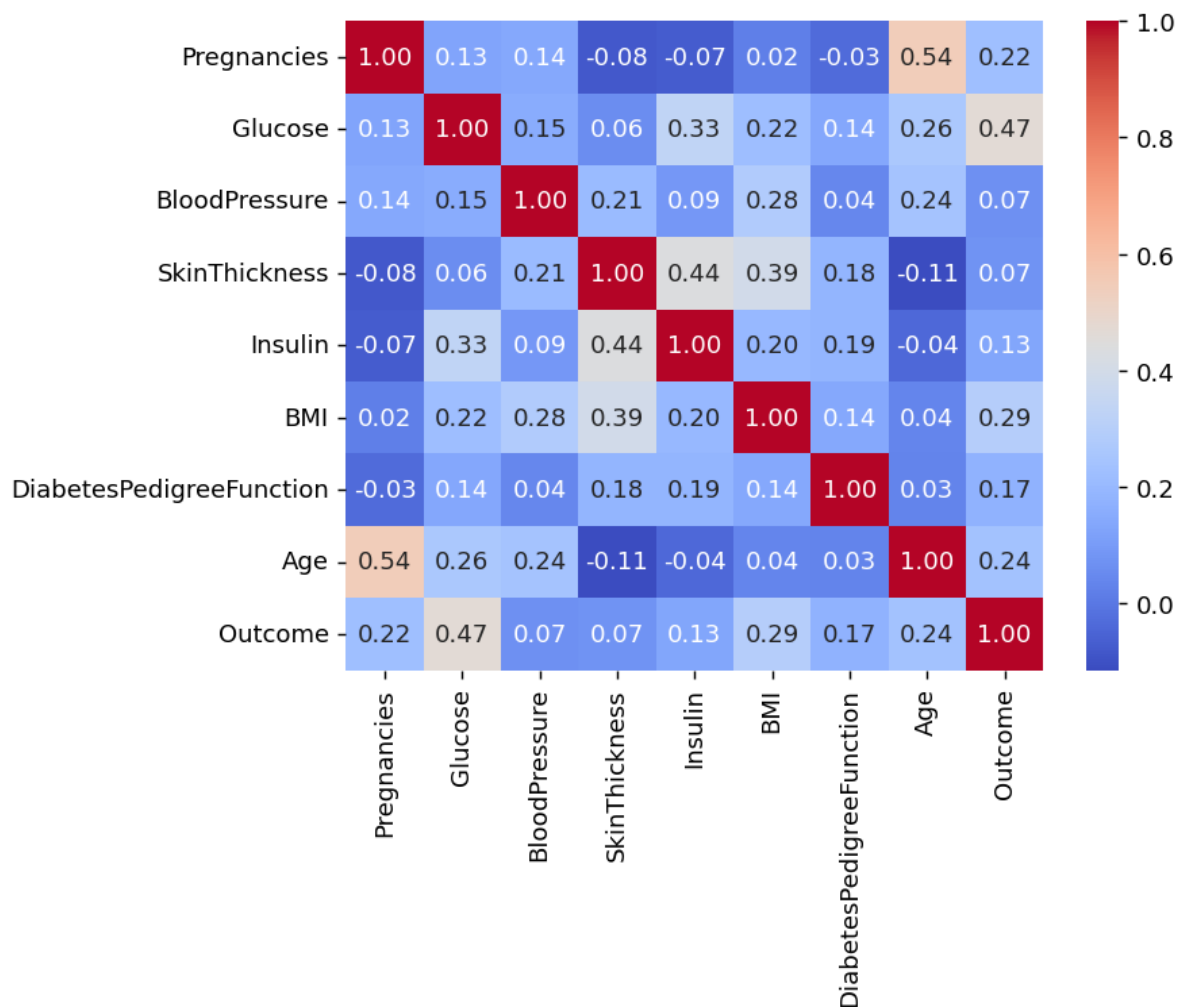




```
In [9]: q1, q3 = np.percentile(df["Insulin"], [25, 75])
        iqr = q3 - q1
        lower = q1 - 1.5 * iqr
        upper = q3 + 1.5 * iqr
        clean_df = df[(df["Insulin"] >= lower) & (df["Insulin"] <= upper)]
```

```
In [10]: corr = df.corr()
         plt.figure(dpi=130)
         sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm")
         plt.show()

         print(corr["Outcome"].sort_values(ascending=False))
```



```

Outcome          1.000000
Glucose          0.466581
BMI              0.292695
Age              0.238356
Pregnancies      0.221898
DiabetesPedigreeFunction 0.173844
Insulin          0.130548
SkinThickness    0.074752
BloodPressure    0.065068
Name: Outcome, dtype: float64

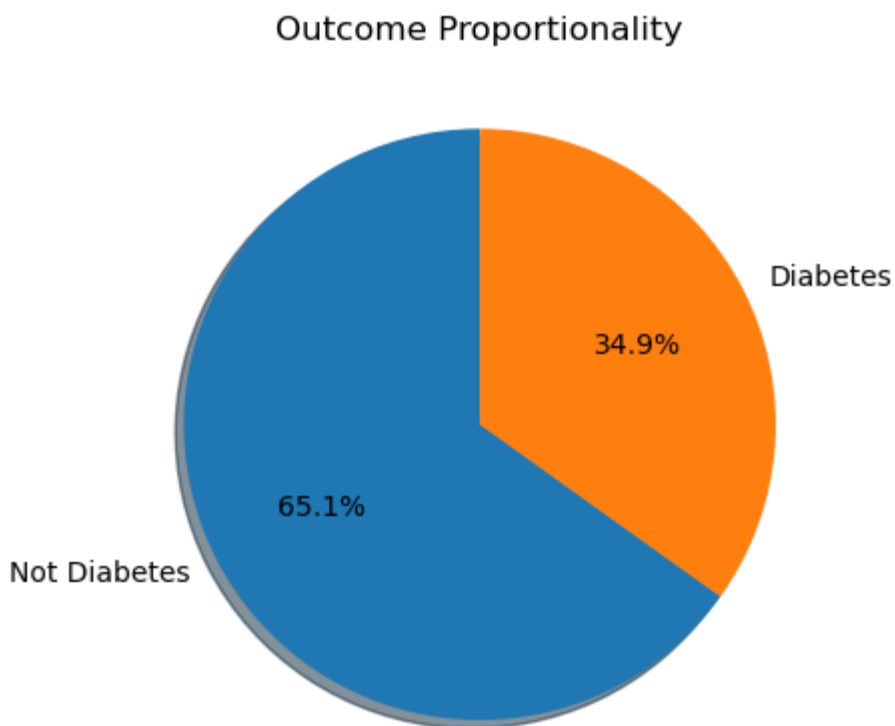
```

```

In [11]: counts = df['Outcome'].value_counts().sort_index()

plt.pie(
    counts,
    labels=['Not Diabetes', 'Diabetes'],
    autopct='%.1f%%',
    shadow=True,
    startangle=90
)
plt.title('Outcome Proportionality')
plt.show()

```



```
In [12]: X = df.drop(columns=['Outcome']) # features
         y = df['Outcome']               # target
```

```
In [13]: scaler = MinMaxScaler()
         X_normalized = scaler.fit_transform(X)
         print(X_normalized[:5])
```

```
[[0.35294118 0.74371859 0.59016393 0.35353535 0.          0.50074516
  0.23441503 0.48333333]
 [0.05882353 0.42713568 0.54098361 0.29292929 0.          0.39642325
  0.11656704 0.16666667]
 [0.47058824 0.91959799 0.52459016 0.          0.          0.34724292
  0.25362938 0.18333333]
 [0.05882353 0.44723618 0.54098361 0.23232323 0.11111111 0.41877794
  0.03800171 0.          ]
 [0.          0.68844221 0.32786885 0.35353535 0.19858156 0.64232489
  0.94363792 0.2          ]]
```

```
In [14]: scaler = StandardScaler()
         X_standardized = scaler.fit_transform(X)
         print(X_standardized[:5])
```

```
[[ 0.63994726  0.84832379  0.14964075  0.90726993 -0.69289057  0.20401277
  0.46849198  1.4259954 ]
 [-0.84488505 -1.12339636 -0.16054575  0.53090156 -0.69289057 -0.68442195
 -0.36506078 -0.19067191]
 [ 1.23388019  1.94372388 -0.26394125 -1.28821221 -0.69289057 -1.10325546
  0.60439732 -0.10558415]
 [-0.84488505 -0.99820778 -0.16054575  0.15453319  0.12330164 -0.49404308
 -0.92076261 -1.04154944]
 [-1.14185152  0.5040552  -1.50468724  0.90726993  0.76583594  1.4097456
  5.4849091  -0.0204964 ]]
```

In []: